



**UNIVERZITET U NOVOM SADU  
FAKULTET TEHNIČKIH NAUKA**



**UNIVERZITET U NOVOM SADU  
FAKULTET TEHNIČKIH NAUKA  
NOVI SAD**

**Departman za računarstvo i automatiku  
Odsek za računarske upravljačke sisteme**

## **PROJEKTNI RAD**

**Kandidat: Luka Mihajlović**

**Broj indeksa: RA 43/2022**

**Predmet: Softverski algoritmi u sistemima automatskog upravljanja**

**Tema rada: Model mašinskog učenja za predviđanje broja  
iznajmljenih bicikala**

**Novi Sad, septembar, 2025.**

# Sadržaj

|                                             |    |
|---------------------------------------------|----|
| Uvod.....                                   | 3  |
| Opis skupa podataka .....                   | 4  |
| Eksplorativna analiza podataka.....         | 5  |
| Izbor, treniranje i testiranje modela ..... | 7  |
| Metrike i vizualizacija.....                | 8  |
| <i>Project.py</i> .....                     | 9  |
| Rezultati i diskusija.....                  | 10 |

# Uvod

Cilj ovog projekta je pravljenje modela mašinskog učenja koji predviđa potražnju za iznajmljivanjem bicikala u odnosu na doba godine i dana, kao i na vremenske uslove. Pošto broj iznajmljenih bicikala zavisi od raznih faktora, ovaj sistem je pogodan za pravljenje, treniranje i evaluaciju modela mašinskog učenja i njegov značaj je u tome što može pomoći u optimizaciji raspodele resursa i poboljšanju kvaliteta usluge. Program za ovaj model je pisan u programskom jeziku *Python*. Generalno govoreći, projekat se sastoji iz nekoliko delova/faza, a to su:

- Analiza datog skupa podataka i diskusija o tome kako obraditi iste kako bi se postigla što veća preciznost i efikasnost modela
- Eksplorativna analiza podataka – statistička obrada i vizualizacija podataka kako bi se oni spremili za treniranje i testiranje modela
- Primenjivanje više modela tj. algoritama mašinskog učenja
- Treniranje i testiranje modela
- Analiza dobijenih podataka pomoću određenih metrika i vizualizacija

# Opis skupa podataka

Izvor skupa podataka koji se obrađuju je fajl *hour.csv* koji sadrži:

- Ulazne podatke:
  - Indikatore rednog broja jednog kompletnog podatka<sup>1</sup>:
    - *instant* - jedinstveni redni ID za svaki zapis
    - *dteday* - datum iznajmljivanja
    - *season* - sezona u kojoj se iznajmljivanje dogodilo
    - *yr* - godina iznajmljivanja
    - *mnth* - mesec iznajmljivanja
    - *hr* - sat iznajmljivanja
  - Kao i karakteristike tog vremenskog trenutka:
    - *holiday* - da li je dan bio praznik
    - *weekday* - dan u nedelji (broj od 0 do 6)
    - *workingday* - da li je dan radni dan
  - Podatke o vremenskim uslovima:
    - *weathersit* - vremenski uslovi (kategorizovano)
    - *temp* - temperatura
    - *atemp* - prilagođena temperatura
    - *hum* - vlažnost vazduha
    - *windspeed* - brzina vetra
- Izlazne podatke:
  - *casual* - broj korisnika koji nisu prethodno registrovani
  - *registered* - broj registrovanih korisnika
  - *cnt* - ukupan broj iznajmljivanja

Sa obzirom na to da se iz podataka vidi da je *cnt* u direktnoj korelaciji sa *casual* i *registered* (njihov zbir), odlučeno je da se projekat podeli na dva dela tj. opcije, da model predviđa samo *cnt* i zanemaruje *casual* i *registered* ili da model predviđa *casual* i *registered* od kojih se lako, po potrebi, dobija *cnt*.

---

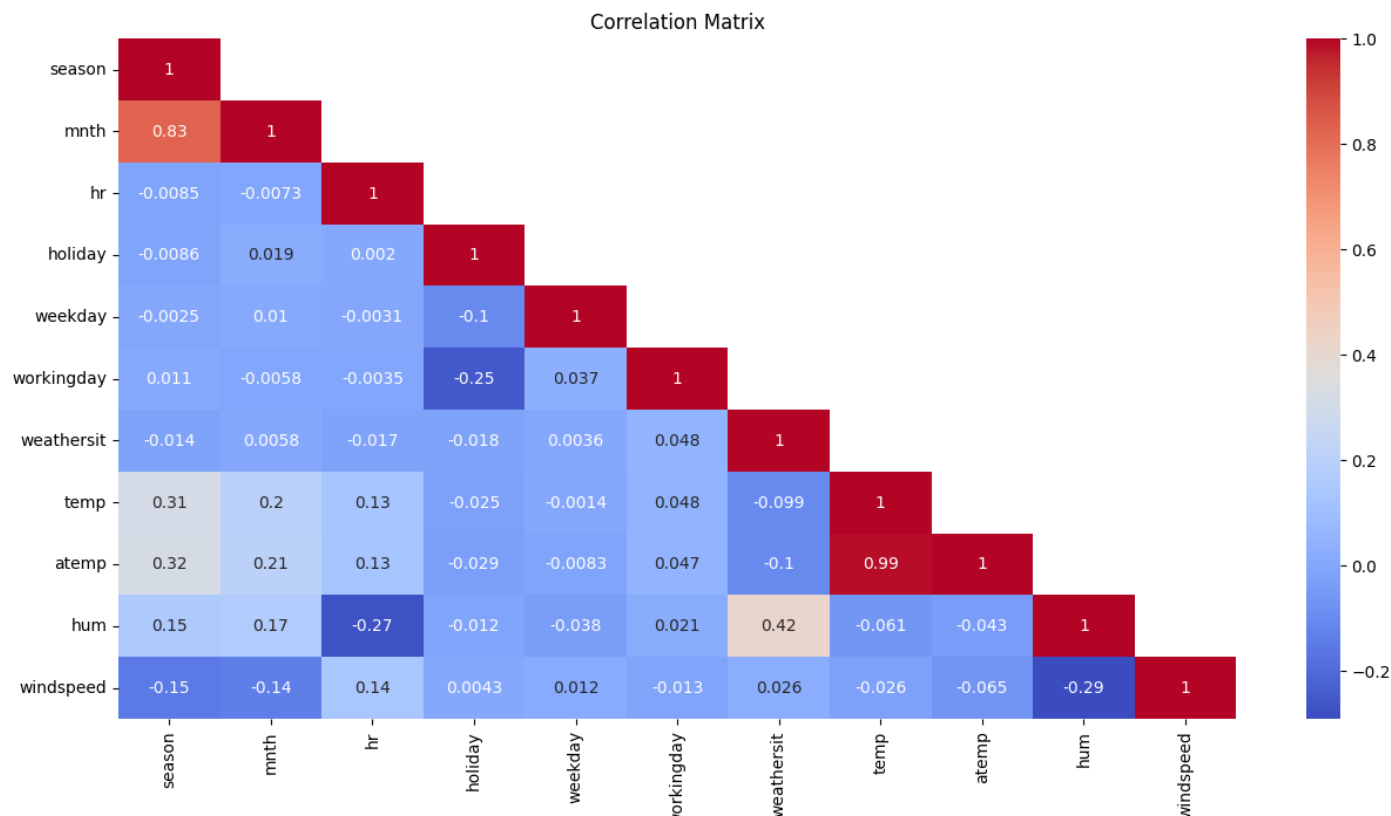
<sup>1</sup> Pod kompletnim podatkom se misli na ceo jedan red ulaznog fajla sa svim njegovim ulaznim i izlaznim podacima, znači jedna instanca skupa.

# Eksplorativna analiza podataka

Eksplorativna analiza podataka odrađena je u fajlu *EDA.py* i u samom početku je podeljena na dve metode, u zavisnosti da li želimo da model predviđa samo *cnt* ili *casual* i *registered*, *cnt\_model()* i *cas\_reg\_model()*. Ove dve metode se po postupku ne razlikuju mnogo, a on je:

- Učitavanje podataka iz ulaznog fajla *hour.csv*
- Analiza i obrada nad celim skupom podataka – metoda *\_eda\_ds()*
  - Uklanjanje duplikata redova – ugrađena funkcija *drop\_duplicates()*
  - Iako se na osnovu broja redova ulaznog fajla (koji je manji od 731 (dve godine) \* 24 (sata)) vidi da postoje redovi koji nedostaju, taj broj je manji od 1% ukupnog skupa podataka pa možemo zanemariti ove nedostatke pošto neće mnogo uticati na kvalitet modela. Mogu da se insertuju nedostajući redovi, ulazne vrednosti se interpoliraju, a izlazne se ostave da bi se koristile za predviđanje, ovde nije potrebno
  - Otkrivanje i obrada nedostajućih vrednosti – metoda *\_fill\_nan()*. U ovoj metodi se nedostajući podaci popunjavaju ili na osnovu pozicije u odnosu na druge podatke u istoj koloni (na primer ako je vrednost pre nedostajuće vrednosti *hr* bila 22, onda se popunjava sa 23) ili na osnovu prosečne vrednosti ostatka kolone (*temp*, *atemp*, *hum* i *windspeed*). U slučaju da je nedostajuća vrednost u nekoj od izlaznih kolona (*casual*, *registered* i *cnt*), taj ceo red se izbacuje iz skupa jer nema smisla interpolirati izlazne podatke jer njih treba da predviđamo.
  - Kodiranje podataka – sa obzirom da su sve kolone osim *dteday* brojne vrednosti, jedino ima smisla da se samo ta kolona kodira – ugrađena funkcija *to\_datetime()*
  - Otkrivanje i uklanjanje anomalija – metode *\_handle\_anomalies\_nat()* koja uklanja kolone u kojoj su vrednosti koje nemaju fizičkog smisla (na primer ako je *hr* 25) i *\_handle\_anomalies\_z()* koja uklanja kolone u kojoj su vrednosti koje mnogo odstupaju od ostataka koristeći *z-score*.
- Izbacivanje parametara koji očigledno uopšte ne utiču na formiranje izlaza ili onih čije se značajne karakteristike opisuju drugim parametrima (na primer ako je datum bio za vreme zime, to će biti opisano vremenskim prilikama ili ako je bio praznik videće se na osnovu *holiday*). Takođe, izbacuju se suvišni izlazi – *casual* i *registered* za *cnt\_model()*, a *cnt* za *cas\_reg\_model()*
- Podela na ulazne i izlazne podatke koje ove metode vraćaju kada bivaju pozvane
- Obrada ulaznih podataka – metoda *\_eda\_X()*
  - Nije potrebno skalirati podatke, pošto su svi bitni podaci (koji su sada preostali) već skalirani

- Otkrivanje i obrada korelacija – vizualizacija pomoću *heatmap* na osnovu koje se vidi da su *temp* i *atemp* u visokoj korelaciji 0.99, isto se vidi i za *season* i *mnth* 0.83 tako da mozemo da izbacimo na primer *atemp* i *season*



# Izbor, treniranje i testiranje modela

Ova faza je odrađena u fajlovima *LR\_Ridge\_Lasso.py* i *DT\_Ensemble.py*. Pošto je potrebno da model izračuna konkretnu vrednost (*cnt*, *causal*, *registered*) u pitanju je regresioni problem, tako da je odlučeno da su modeli tj. algoritmi koji će biti upotrebljeni Linearna, *Ridge* i *Lasso* regresija, kao i regresioni oblici Stabla odlučivanja (*Decision Tree*) i ansambl metoda *Random Forest* i *Gradient Boosting*. Nad svim izabranim modelima izvršena je unakrsna validacija i podešavanje hiperparametara, ako isti postoje. Takođe, svi modeli su prilagođeni da predviđaju kako jednodimenzionu, tako i višedimenzionu izlaz, što je neophodno zbog prethodno pomenute podele na osnovu toga koje izlazne parametre želimo da model predviđa.

Što se tiče konkretnih modela, svi prate sličan postupak:

- Model linearne regresije prvo proverava dimenzionalnost izlaza kako bi znao kako da definiše sam model u kodu, ako je u pitanju jednodimenzionu izlaz koristimo regularnu funkciju *LinearRegression()*, ako je ipak višedimenzionu izlaz, onda model mora da se “obmota” ugrađenom funkcijom *MultiOutputRegressor()* koja čini da model predviđa višedimenzionu izlaz. Zatim se vrši unakrsna validacija, potom treniranje modela i predviđanje izlaza, na kraju se svi dobijeni podaci šalju metodi *lr\_ridge\_lasso\_metric()* koja ispisuje metrike datog modela. I za *Ridge* i za *Lasso* postupak je isti, samo što kod njih uvodimo optimizaciju hiperparametra *alpha* pomoću ugrađene funkcije *GridSearchCV()*, uzimamo najbolji model i pomoću njega vršimo predviđanje.
- Kod *DT*, *RF* i *GD* prvo definišemo skup hiperparametara koje želimo da istestiramo i podesimo, zatim sledi unakrsna validacija i podešavanje hiperparametara, zatim uzimamo najbolji model i nad njime vršimo predviđanje i izvlačimo najbolje vrednosti hiperparametara *best\_params\_* i koeficijent važnosti ulaznih podataka *feature\_importances\_*. Zatim pripremamo podatke za vizualizaciju zavisnosti određenih stvari kod modela, kod *DT* to je koeficijent orezivanja *ccp\_alpha* i dubine stabla, *ccp\_alpha* i kvaliteta modela, kod *RF* to je broj stabala *n\_estimators* i kvalitet modela, kod *GB* to je broj stabala *n\_estimators* i kvalitet modela, brzina učenja *learning\_rate* i kvalitet modela. Na kraju sve podatke šaljemo metodi *dt\_rf\_gb\_metric()* koja ispisuje metrike i pravi grafikone zavisnosti. Jedino se mora voditi računa da, za razliku od *DT* i *RF*, *GB* nema sam po sebi mogućnost da predviđa višedimenzionu izlaz, tako da moramo da ga “obmotamo” funkcijom *MultiOutputRegressor()*.

# Metrike i vizualizacija

Ispisivanje metrika i vizualizacija grafikona se obavlja u fajlu *Visualization.py*.

- Što se tiče Linearne, *Ridge* i *Lasso* regresije, samo se izračunavaju najosnovnije metrike - srednja kvadratna greška (*Mean Squared Error* - *MSE*), kvadratni koren srednje kvadratne greške (*Root Mean Squared Error* - *RMSE*), srednja apsolutna greška (*Mean Absolute Error* - *MAE*) i  $R^2$  score.  $R^2$  score će biti glavna metrika nadalje, pogotovo u graphicima. Ove metrike, zajedno sa parametrima kao što su *MSE* unakrsne validacije, najboljeg *alpha* i koeficijenta ulaznih parametara kod *Ridge* i *Lasso* se ispisuju na terminal.
- Kod *DT*, *RF* i *GB* se računaju i ispisuju iste metrike, ali se pored toga i generišu grafikoni zavisnosti određenih karakteristika modela. Promenljiva *figs* služi da se svi *plot*-ovi sačuvaju u listu, kako bi se na samom kraju programa generisali da ne bi zamrzavali program, a opet imali dovoljno vremena da se *render*-uju. U zavisnosti od modela se generišu različiti grafikoni:
  - *DT* – grafikon zavisnosti dubine stabla od koeficijenta orezivanja *ccp\_alpha*, grafikon zavisnosti  $R^2$  score od koeficijenta orezivanja *ccp\_alpha*, kao i generisanje Stabla odlučivanja (do dubine 3 zato što se za veće dubine ništa ne može da se pročita sa slike)
  - *RF* – grafikon zavisnost  $R^2$  score od broja stabala *n\_estimators*
  - *GB* – grafikon zavisnosti  $R^2$  score od broja stabala *n\_estimators* i grafikon zavisnosti  $R^2$  score od brzine učenja *learning\_rate*



## *Project.py*

*Project.py* je glavni fajl ovog projekta i iz njega se pozivaju svi ostali fajlovi tj. metode iz istih. Ovde se nalazi i jedina interakcija sa korisnikom, a to je izbor rada programa u odnosu na željeni izlaz koju biramo tako što prosleđujemo određeni broj glavnoj metodi *run()*:

0 – ako želimo da model predviđa *cnt* i zanemaruje *casual* i *registered* (*default*)

1 – ako želimo da model predviđa *casual* i *registered*

2 – ako želimo da uradi oba da imamo da poredimo na jednom mestu

Takođe je implementirana metoda *measure\_time()* koja meri i ispisuje trajanje izvršavanja svakog pozvanog modela kako bi nam omogućilo lakše praćenje vremenske efikasnosti modela da bismo mogli da uzmemo to u obzir pri pokretanju određenih modela (pogotovo *RF* i *GB*) i odabiru količine hiperparametara koje želimo da podesimo. Na kraju program ispisuje trajanje izvršavanja celog programa i generiše sve grafikone.

## Rezultati i diskusija

Sve izlazne metrike koje se prikazuju na terminalu se nalaze u tekstualnom fajlu *Metrics.txt* a u prilogu su i slike svih *plot*-ovanja ovog programa. Prvo ćemo se osvrnuti na metrike koje su definisane za sve modele radi upoređivanja, a zatim ćemo se osvrnuti na pojedine aspekte modela samih za sebe.

- Za poređenje različitih modela nema potrebe razmatrati sve metrike posebno, dovoljno je jednu tako da kao glavnu metriku za poređenje modela uzimamo  $R^2$  score jer je nareprezentativniji, a pritom poprilično intuitivan.  $R^2$  score i vreme izvršavanja svih modela su redom:
  - Linearna regresija za jednodimenzioni izlaz - 0.326878, 0.0338 sek
  - Ridge regresija za jednodimenzioni izlaz - 0.326876, 0.0827 sek
  - Lasso regresija za jednodimenzioni izlaz - 0.326818, 0.4037 sek
  - Decision Tree za jednodimenzioni izlaz - 0.818052, 1 min 52 sek
  - Random Forest za jednodimenzioni izlaz - 0.855568, 7 min 29 sek
  - Gradient Boosting za jednodimenzioni izlaz - 0.872364, 6 min 59 sek
  - Linearna regresija za višedimenzioni izlaz - 0.344137, 0.0462 sek
  - Ridge regresija za višedimenzioni izlaz - 0.344141, 0.1867 sek
  - Lasso regresija za višedimenzioni izlaz - 0.344137, 0.7833 sek
  - Decision Tree za višedimenzioni izlaz - 0.806647, 2 min 5 sek
  - Random Forest za višedimenzioni izlaz - 0.868561, 8 min 1 sek
  - Gradient Boosting za višedimenzioni izlaz - 0.879136, 14 min 7 sek

Na osnovu ovoga se izvodi nekoliko zaključaka:

- Modeli za višedimenzioni izlaz se ne razlikuju mnogo od onih za jednodimenzioni izlaz, vidi se blagi trend porasta  $R^2$  score ali to je uglavnom tek na drugoj decimali najviše, što je i donekle očekivano sa obzirom da su *casual*, *registered* i *cnt* u direktnoj korelaciji.
- $R^2$  score između Linearne, Ridge i Lasso regresije se praktično ne razlikuje, tek na šestoj decimali, što nam govori da Ridge i Lasso nemaju veliki uticaj na skup podataka što znači da je on prethodno dobro obrađen i nema jakih korelacija između atributa.
- $R^2$  score DT, RT i GB je daleko bolji od Linearne, Ridge i Lasso regresije, što je opet očekivano sa obzirom da oni mnogo bolje modeluju nelinearne zavisnosti, koje su u datom skupu podataka definitivno prisutne, naravno uz mnogo duže izvršavanje.
- Ansambl metode dodatno poboljšavaju performanse u odnosu na DT, pritom im je vreme izvršavanja višestruko duže, tako da se mora pažljivo birati nad kojim skupom hiperparametara se vrši optimizacija, sa obzirom da to zauzima najviše procesorskog vremena.

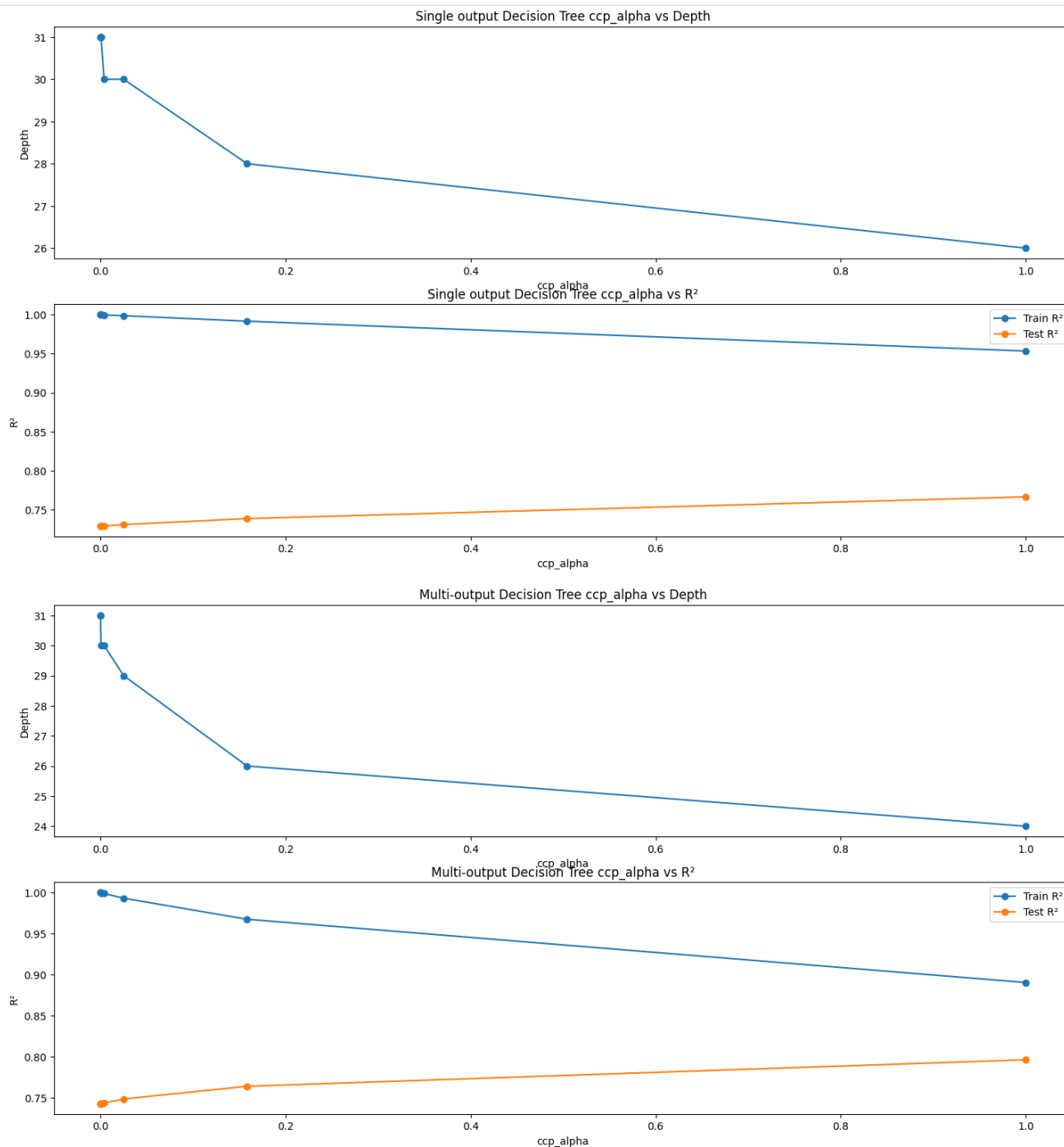
- Što se tiče modela samih za sebe možemo prokomentarisati:
  - *CV MSEs*: [7424.621149 7393.80086491 7323.38505737 7498.03406509 7183.17391238] – srednje kvadratne greške različitih kombinacija podataka za treniranje pri unakrsnoj validaciji. Sve vrednosti su u relativno bliskom opsegu, što znači da je model poprilično robustan i nema overfitting-a.
  - *Best alpha* – najbolje *alpha* (jedini hiperparametar) kod *Ridge* i *Lasso*.
  - *Coefficients* – koeficijenti svih atributa skupa podataka:
    - Linearna regresija : [ 4.29316268 7.08364714 -25.50693632 2.01095427 -5.48478221 -4.11023179 280.74847572 -202.77326959 7.32315894]
    - *Ridge* : [ 4.29532358 7.08948192 -25.44148369 2.01313025 -5.46355624 -4.19789302 280.16732789 -202.20683152 7.50699693]
    - *Lasso* : [ 4.26458542 7.12099001 -21.34317869 2.0243028 -4.63142086 -4.1402066 277.9549724 -200.95820134 1.06692906]

Pošto se koeficijenti ne razlikuju mnogo za tri modela, to vodi istom zaključku iz prethodnog dela diskusije - *Ridge* i *Lasso* nemaju veliki uticaj na skup podataka što znači da je skup podataka prethodno solidno obrađen i nema jakih korelacija između atributa.

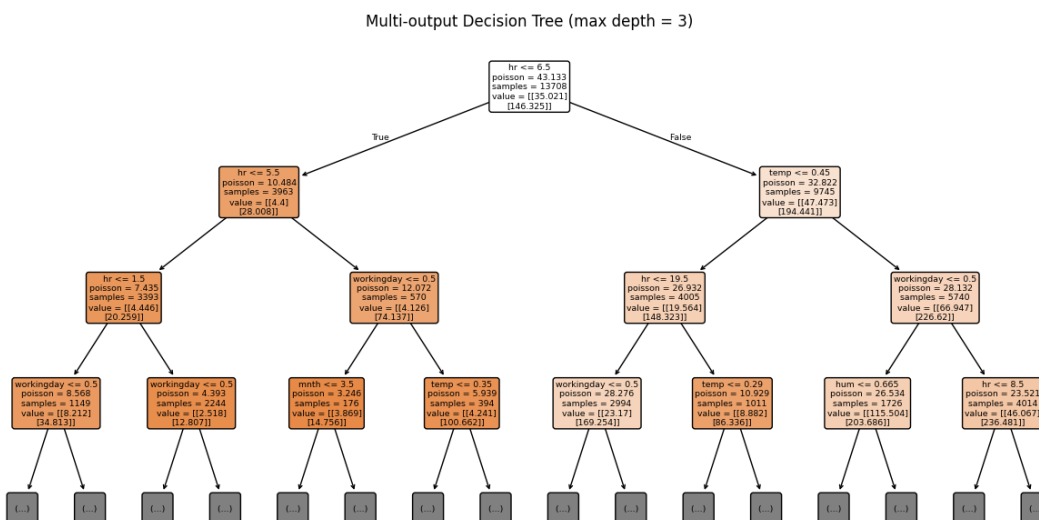
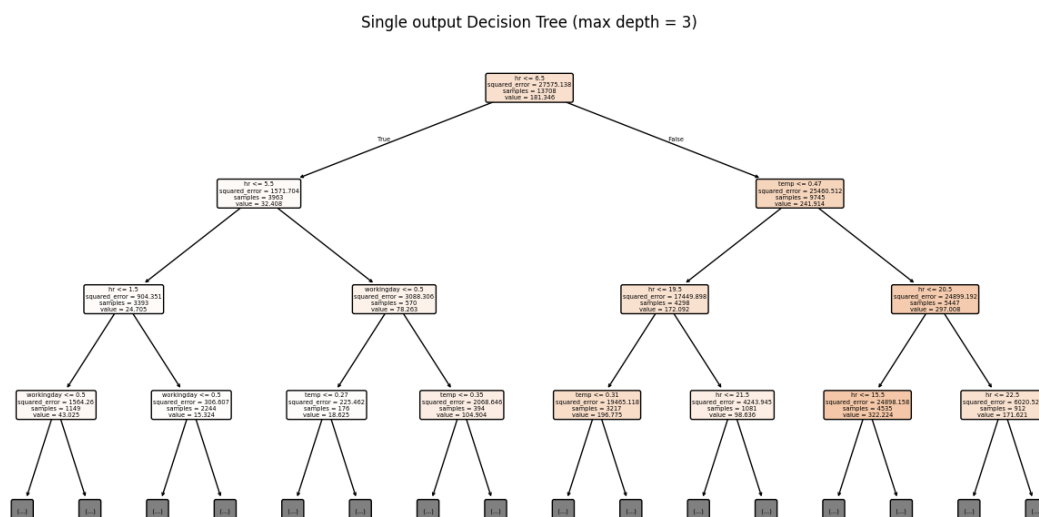
- *Best parameters* – najbolje vrednosti različitih hiperparametara koji su bili podešavani kod *DT*, *RF* i *GB*.
- *Feature importances* – pokazuje koliko je svaki od atributa skupa podatka važan tj. koliko utiče na izlazne podatke, prikazano je za *DT*, *RF* i *GB*. Vrednosti su slične za sva tri pa ćemo uzeti za primer samo jedan:
  - mnth: 0.0670
  - hr: 0.5196
  - holiday: 0.0037
  - weekday: 0.0418
  - workingday: 0.0339
  - weathersit: 0.0242
  - temp: 0.1518
  - hum: 0.1106
  - windspeed: 0.0474

Iz ovoga se vidi da je daleko najvažniji atribut satnica iznajmljivanja bicikla, ali se takođe ističu i temperatura i vlažnost vazduha.

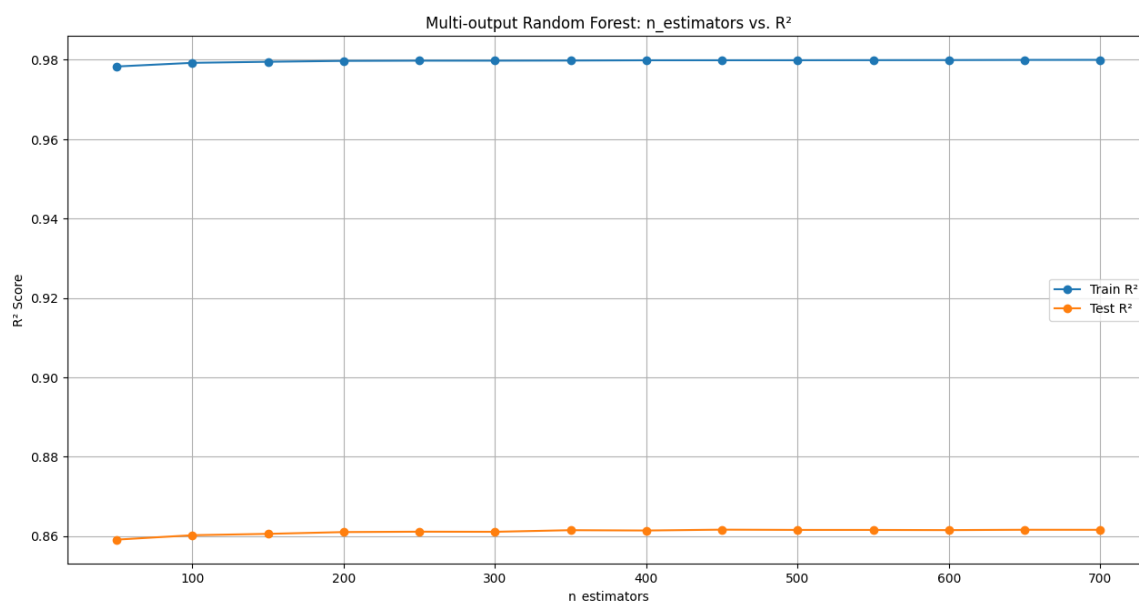
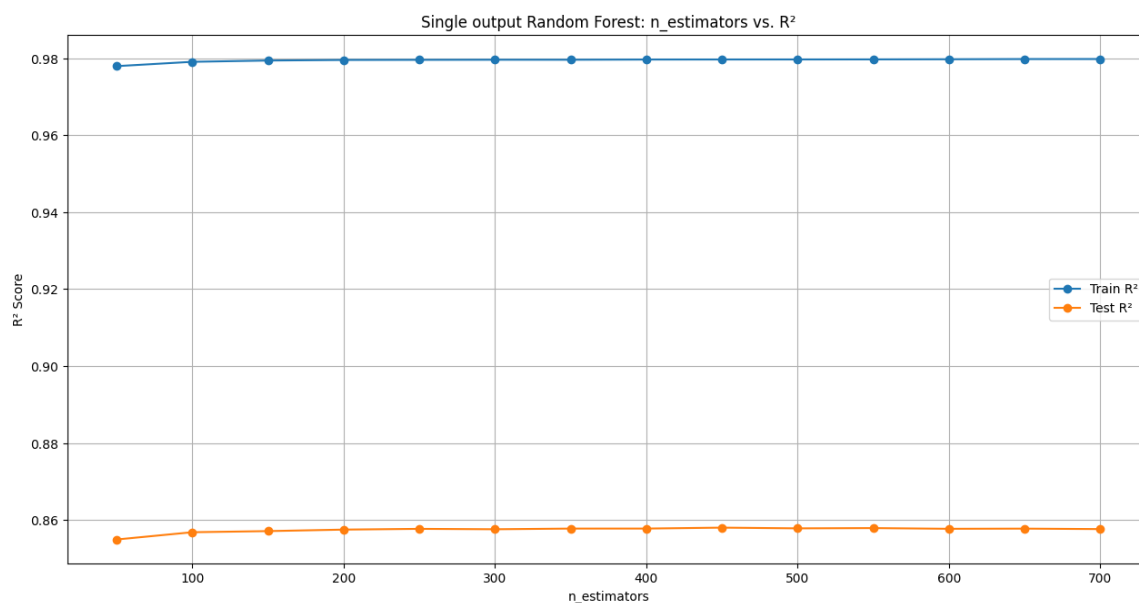
- *DT ccp\_alpha vs depth* i *ccp\_alpha vs R<sup>2</sup> score* – grafikoni nam pokazuju zavisnost dubine stabla i kvaliteta modela u odnosu na koeficijent orezivanja. Vidi se da je trend sličan i za jednodimenzionu i za višedimenzionu izlaz, za veće koeficijent orezivanja dubina stabla je manja a kvalitet je malo veći.



- *DT plotted* – prikaz stabla odlučivanja tj. kako ono izgleda i kako donosi odluke na osnovu kojih pitanja i kojih parametara, prikaz ide samo do dubine 3 zato što se za veće dubine ne može ništa razaznati sa slike.



- *RF*  $n\_estimators$  vs  $R^2$  score – grafikoni nam pokazuju zavisnost kvaliteta modela od broja stabala koja se generišu. Opet se vidi isti trend i kod jednodimenzionog i kod višedimenzionog izlaza a to je da kada je broj generisanih stabala u stotinama, nema značajne promene kvaliteta modela, a treba više vremena da se on istrenira tako da nema potrebe za vrednosti  $n\_estimators$  većih od 100-200.



- *GB n\_estimators vs  $R^2$  score i learning\_rate vs  $R^2$  score* – grafikoni nam prikazuju zavisnost kvaliteta modela od broja stabala koja se generišu i od brzine učenja. Još jednom se vidi isti trend i kod jednodimenzionog i kod višedimenzionog izlaza a to je da se kvalitet ustaljuje za broj stabala oko 400-500 što znači da nema potrebe dalje žrtvovati vremensku efikasnost radi jako malih poboljšanja kvaliteta modela i kvalitet je najveći za brzinu učenja oko 0.3-0.5, za veće vrednosti ne samo da kvalitet počinje blago da opada već se i značajno povećavaju šanse za overfitting.

